



Université Paris Cité

PROJET DE DEEP-LEARNING

Classification du Carcinome Canalaire Infiltrant (IDC)

ELIE Kouldjim & PALESSONGA Gedeon Esaie & TITO Rytrace

MASTER II IMB

1 Introduction générale

2 I. Methodologie

3 II. Resultats

4 Conclusion

Contexte et problématique

- Le cancer du sein est le plus fréquent chez les femmes
- Le Carcinome Canalaire Infiltrant (IDC) est la forme la plus courante.
- Le diagnostic visuel par les pathologistes est critique, mais long et sujet à la variabilité inter-observateur.
- Les modèles de deep learning peuvent aider au diagnostic assisté

Objectif generale et specifiques

- Construire un modèle de classification d'images médicales
 - Un modèle capable d'identifier les tissus IDC des tissus non IDC
 - Evaluer les performances du modèle
 - Proposer les pistes d'amélioration

1 Introduction générale

2 I. Methodologie

3 II. Resultats

4 Conclusion

Demarche analytique

- Analyse exploratoire pour decrire les données
- Préparation des données:
 - Division des données en ensemble d'entraînement, test et validation
 - Creation des ensembles des données PyTorch contenant des images et des étiquettes
- Construction des modèles de réseau de neurones et utilisation des modèles prédéfinis:
 - Trois modèles de reseau de neurones convolutifs (CNN)
 - Deux modèles prédéfinis: RESET18 et VGG
 - Un modèle Transformer VIT

Demarche analytique

- Métriques d'Évaluation Cruciales: Tableau de Performance Final (Synthèse)
- Interpretation en utilisant l'analyse Heatmap Grad-CAM

1 Introduction générale

2 I. Methodologie

3 II. Resultats

4 Conclusion

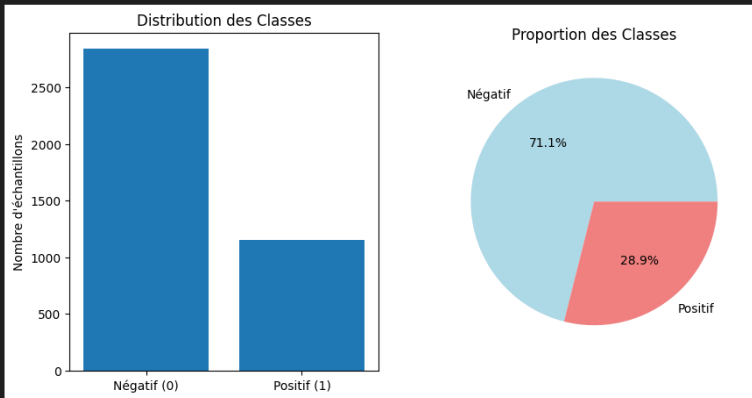
II.1 Analyse Exploratoire

Analyse exploratoire

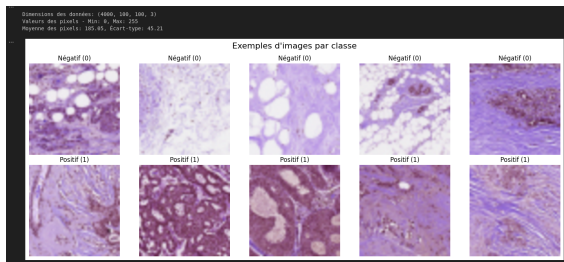
- Images de tissu mammaires, découpés en patches
- 4000 images choisies aléatoirement dont ceux comprenant les IDC et les non IDC
- Sur l'échantillon aléatoire de 4000 images, nous observons un déséquilibre de classes significatif.
- La classe 0 (IDC Négatif), qui représente les zones de tissu sans cancer canalaire infiltrant, constitue environ 72.2
- La classe 1 (IDC Positif), qui représente les zones avec cancer, ne représente qu'environ 27.8

II.1 Analyse Exploratoire

```
... Distribution des classes:  
Classe 0 (Négatif): 2843 échantillons  
Classe 1 (Positif): 1157 échantillons  
Ratio Positif/Négatif: 0.41
```



II.1 Analyse Exploratoire



Les images de tissus sains (“Négatif - 0”) sont généralement claires et ordonnées. Elles contiennent souvent des grandes zones blanches (graisse) et les petits points violets (noyaux des cellules) sont rares et bien espacés. Le tissu a une structure normale. À l’inverse, les images de tissus cancéreux (“Positif - 1”) sont foncées et chaotiques. Elles sont envahies par des amas très denses et irréguliers de points violets foncés. Ces amas représentent la masse des cellules cancéreuses qui se multiplient et envahissent le tissu. Le modèle de deep learning doit donc apprendre à distinguer l’ordre (Négatif) du chaos cellulaire dense (Positif).

II.2 Presentation de la construction des données d'entrainement et données de test

Division Stratifiée des Données

- Objectif : Diviser l'échantillon aléatoire de 4000 images en trois ensembles distincts pour garantir une évaluation non biaisée.
- Ratios de Division :
 - Entraînement (Train) : $\sim 70\%$ des données (utilisé pour l'apprentissage des poids).
 - Validation (Validation) : $\sim 15\%$ des données (utilisé pour l'ajustement des hyperparamètres).
 - Test (Test) : $\sim 15\%$ des données (utilisé pour l'évaluation finale des performances).
- Méthode Cruciale : Stratification
 - L'algorithme a utilisé la stratification pour garantir que la distribution initiale des classes.

II.4 Architecture du CNN1

```
class SimpleCNN1(nn.Module):
    def __init__(self, num_classes=2, in_channels=3):
        super(SimpleCNN1, self).__init__()

        # Couches convolutives
        self.conv1 = nn.Conv2d(in_channels, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)

        # Pooling
        self.pool = nn.MaxPool2d(2, 2)

        # POOLING ADAPTATIF - la solution la plus simple
        # Cela redimensionnera automatiquement à 6x6
        self.adaptive_pool = nn.AdaptiveAvgPool2d((6, 6))

        # Maintenant nous savons que la taille sera: 128 * 6 * 6 = 4608
        self.fc1 = nn.Linear(128 * 6 * 6, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))

        # Pooling adaptatif garantit la bonne taille
        x = self.adaptive_pool(x)

        #Application (flatten)
        x = x.view(x.size(0), -1)

        # Couches entièrement connectées
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x
```

II.5 Fonction d'entraînement et résultats du modèle CNN1 entraînée

Rôle de la fonction `train_model_simple`

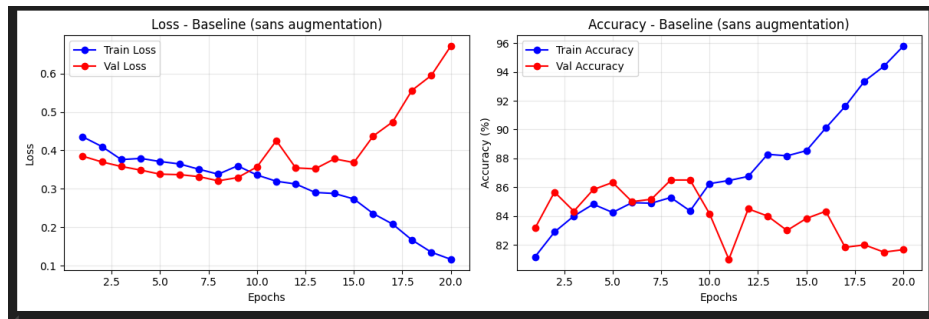
- **Objectif** : Entraînement de modèle sur un nombre fixe d'époques (`num_epochs = 20`), sans mécanismes d'optimisation avancés (*early stopping*).
- **Principe** : Suivi de cycle standard de l'apprentissage profond :
 - propagation avant (*forward pass*),
 - calcul de la fonction de perte,
 - rétropropagation (*backward pass*),
 - mise à jour des poids (*optimizer step*).

II.5 Fonction d'entraînement et résultats du modèle CNN1 entraînée

Hyperparamètres et calculs clés

- **Optimiseur** : Adam avec un taux d'apprentissage $LR = 0.001$, choisi pour sa rapidité de convergence.
- **Fonction de perte** : `nn.CrossEntropyLoss`.
 - Compatible avec une sortie du modèle sous forme de deux logits bruts.
 - Les labels doivent être convertis en `torch.long` avant le calcul de la perte.
- **Métriques suivies** : À chaque époque, la fonction calcule et enregistre :
 - **Train Loss** et **Train Accuracy** (suivi de l'apprentissage),
 - **Validation Loss** et **Validation Accuracy** (détection du surapprentissage).

II.5 Fonction d'entraînement et résultats du modèle CNN1 entraînée



Résultats et Conclusion (CNN1 - Baseline)

- Observation : Avec la fonction d'entraînement simple, le modèle CNN1 a rapidement affiché une divergence entre Train Loss et Val Loss.
- C'est la preuve que, sur des données non équilibrées et sans augmentation, le modèle souffre d'un surapprentissage sévère.

II.6 Architecture du CNN2

```
class SimpleCNN2(nn.Module):
    def __init__(self, num_classes=2, in_channels=3, dropout_rate=0.5):
        super(SimpleCNN2, self).__init__()

        # Couches convolutives
        self.conv1 = nn.Conv2d(in_channels, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)

        # Pooling
        self.pool = nn.MaxPool2d(2, 2)

        # Dropout
        self.dropout = nn.Dropout(dropout_rate)

        # POOLING ADAPTATIF - la solution la plus simple
        # Cela redimensionnera automatiquement à 6x6
        self.adaptive_pool = nn.AdaptiveAvgPool2d((6, 6))

        # Maintenant nous savons que la taille sera: 128 * 6 * 6 = 4608
        self.fc1 = nn.Linear(128 * 6 * 6, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))

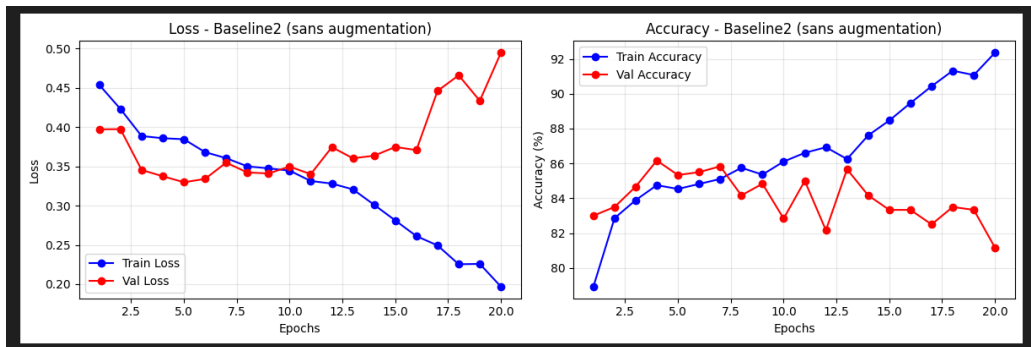
        # Pooling adaptatif garantit la bonne taille
        x = self.adaptive_pool(x)

        # Debug
        # print(f"Shape après adaptive pool: {x.shape}")
        # Devrait être: (batch, 128, 6, 6)

        # Application (flatten)
        x = x.view(x.size(0), -1)

        # Couches entièrement connectées
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
        x = self.dropout(x)
        x = self.fc3(x)
        return x
```

II.7 Resultats du modèle CNN2 entraîné



Résultats et Conclusion (CNN2 - Baseline)

- Toujours le surapprentissage dans le meme cas du CNN1 à cause du déséquilibre des classes observés
- Nécessité d'améliorer les données : Rééquilibrer les classes et augmenter les données

II.8 Processus de Data augmentation et amélioration puis entraînement de CNN2

Pilier 2 : Équilibrage par sous-échantillonnage (lutte contre le biais et surapprentissage)

- **Problème identifié** : La forte proportion de la classe négative (72.2 %) incitait le modèle à prédire majoritairement la classe négative, introduisant un biais important.
- **Méthode utilisée** : Sous-échantillonnage aléatoire strict (*Random Undersampling*).
 - Sélection aléatoire d'un nombre d'images négatives égal à la taille de la classe positive (minoritaire).
- **Résultat obtenu** : Création d'un `train_loader_balanced` présentant une distribution parfaitement équilibrée :

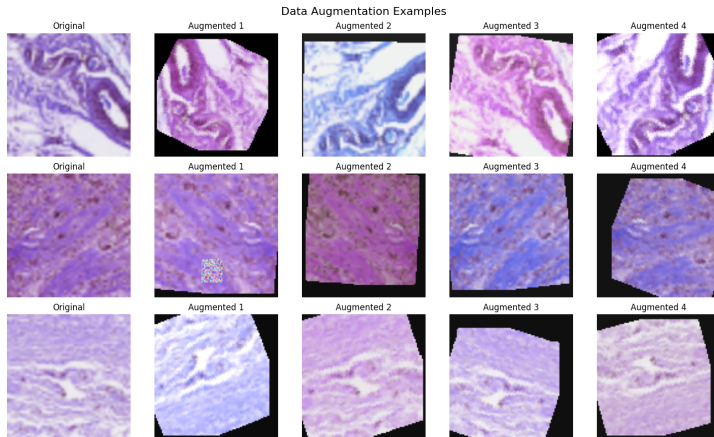
50 % Positif / 50 % Négatif.

II.8 Processus de Data augmentation et amélioration puis entraînement de CNN2

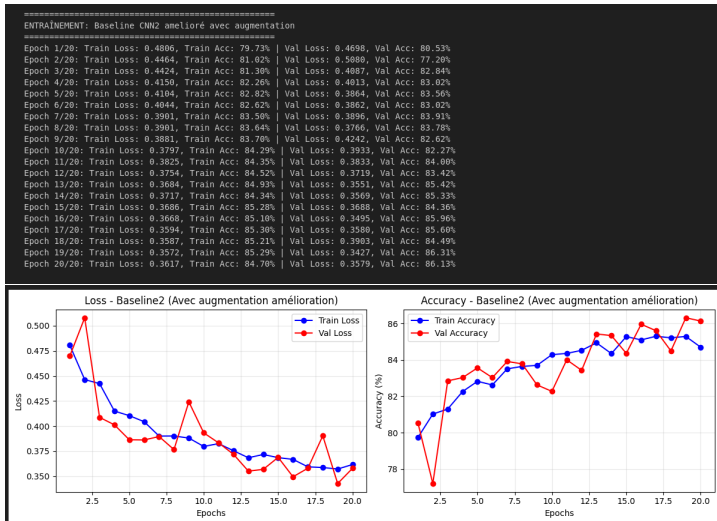
Pilier 1 : Augmentation des données

- **Problème identifié** : Le manque de diversité du jeu d'entraînement initial a conduit à un surapprentissage marqué du modèle CNN1 et CNN2.
- **Nouvelle stratégie** : Le `train_transform` a été enrichi par des transformations aléatoires appliquées à la volée, sans modification des ensembles de validation et de test.
- **Transformations utilisées** :
 - `transforms.RandomHorizontalFlip()` et `transforms.RandomVerticalFlip()` : simulation des rotations possibles des lames au microscope.
 - `transforms.RandomRotation(degrees = 30)` : introduction d'une invariance à la rotation.
 - `transforms.ColorJitter(brightness = 0.2, ...)` : simulation de variations d'éclairage ou de coloration histologique.

II.8 Processus de Data augmentation et amélioration puis entraînement de CNN2



II.8 Processus de Data augmentation et amélioration puis entraînement de CNN2



II.8 Processus de Data augmentation et amélioration puis entraînement de CNN2

Résultats de CNN2 (Architecture CNN1 ré-entraînée)

- Amélioration : L'entraînement de CNN2 sur ces données améliorées montre une convergence plus stable et un écart réduit entre la Train Loss et la Val Loss.
- Ces étapes ont permis de régler le surapprentissage.

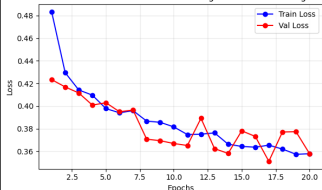
II.9 Construction d'un CNN3

Utilisation d'un simple CNN3 en ajoutant le Batch Normalisation dans l'architecture du modèle

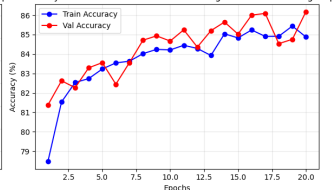
II.10. Entrainement de CNN2 et CNN3 en utilisant l'argument Early Stop

```
=====
ENTRAÎNEMENT: Baseline CNN2 amélioré avec augmentation et earling stop
=====
Learning rate: 0.001
Early stopping patience: 10
Using scheduler: True
=====
Batch 20/165, Loss: 0.4507
Batch 40/165, Loss: 0.3903
Batch 60/165, Loss: 0.4149
Batch 80/165, Loss: 0.4549
Batch 100/165, Loss: 0.6242
Batch 120/165, Loss: 0.4450
Batch 140/165, Loss: 0.4800
Batch 160/165, Loss: 0.3483
Nouveau meilleur modèle: 81.38%
Epoch 1/20: Train Loss: 0.4834, Train Acc: 78.48% | Val Loss: 0.4234, Val Acc: 81.38% | LR: 0.001000 | Patience: 0/10
Batch 20/165, Loss: 0.3556
Batch 40/165, Loss: 0.4231
Batch 60/165, Loss: 0.4192
Batch 80/165, Loss: 0.3901
Batch 100/165, Loss: 0.5009
Batch 120/165, Loss: 0.3663
Batch 140/165, Loss: 0.3283
...
Batch 160/165, Loss: 0.3620
Nouveau meilleur modèle: 86.18%
Epoch 20/20: Train Loss: 0.3579, Train Acc: 84.88% | Val Loss: 0.3579, Val Acc: 86.18% | LR: 0.001000 | Patience: 0/10
Meilleur modèle restauré avec une précision de validation de 86.18%
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings..
```

Loss - Baseline CNN2 amélioré avec augmentation et earling stop

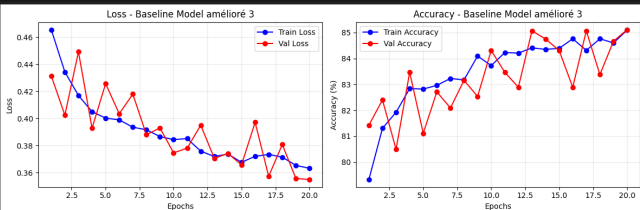


Accuracy - Baseline CNN2 amélioré avec augmentation et earling stop



II.10. Entraînement de CNN2 et CNN3 en utilisant l'argument Early Stop

```
=====
ENTRAÎNEMENT: Baseline Model amélioré 3
=====
Learning rate: 0.001
Early stopping patience: 7
Using scheduler: True
=====
Batch 20/165, Loss: 0.4509
Batch 40/165, Loss: 0.4613
Batch 60/165, Loss: 0.5118
Batch 80/165, Loss: 0.4927
Batch 100/165, Loss: 0.4469
Batch 120/165, Loss: 0.4237
Batch 140/165, Loss: 0.3661
Batch 160/165, Loss: 0.4375
Nouveau meilleur modèle: 81.42%
Epoch 1/20: Train Loss: 0.4654, Train Acc: 79.32% | Val Loss: 0.4314, Val Acc: 81.42% | LR: 0.001000 | Patience: 0/7
Batch 20/165, Loss: 0.4688
Batch 40/165, Loss: 0.4535
Batch 60/165, Loss: 0.3645
Batch 80/165, Loss: 0.2649
Batch 100/165, Loss: 0.4561
Batch 120/165, Loss: 0.4546
Batch 140/165, Loss: 0.3823
...
Batch 160/165, Loss: 0.4393
Nouveau meilleur modèle: 85.11%
Epoch 20/20: Train Loss: 0.3633, Train Acc: 85.10% | Val Loss: 0.3550, Val Acc: 85.11% | LR: 0.001000 | Patience: 0/7
Meilleur modèle restauré avec une précision de validation de 85.11%
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



II.11. Utilisation des modèles predefini RESET18 et VGG puis leurs entraînements

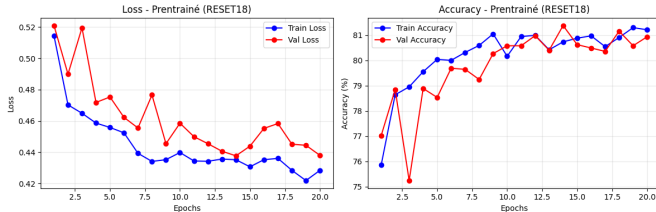
Le transfer learning :principe

- Utilisation d'un ResNet-18 pré-entraîné sur ImageNet
- Réutilisation des caractéristiques visuelles générales
- Adaptation à la classification IDC (2 classes)

Architecture spécifique de PretrainedResNet (adaptation)

- **Extraction des caractéristiques** : Les couches convolutionnelles du corps du réseau sont initialement gelées en fixant `param.requires_grad = False`.
- **Modification de la tête de classification** : La couche finale originale est remplacée par un classificateur adapté à la tâche binaire :
 - **Couches denses intermédiaires** : Ajout de couches fully-connected de dimensions $512 \rightarrow 256 \rightarrow 128$, avec activation ReLU et Dropout (0.5) pour la régularisation.
 - **Sortie finale** : `nn.Linear(128, 2)`, produisant deux logits compatibles avec `CrossEntropyLoss`.

II.11. Utilisation des modèles predefinis RESET18 et VGG puis leurs entrainements.



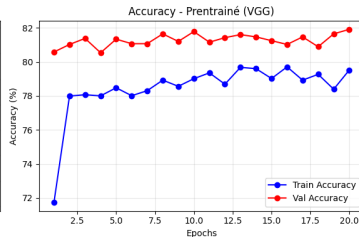
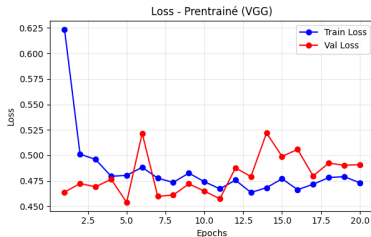
ENTRAÎNEMENT: Baseline Model RESNET18

Epoch 1/20: Train Loss: 0.5144, Train Acc: 75.86%	Val Loss: 0.5210, Val Acc: 77.02%
Epoch 2/20: Train Loss: 0.4703, Train Acc: 78.64%	Val Loss: 0.4899, Val Acc: 78.84%
Epoch 3/20: Train Loss: 0.4649, Train Acc: 78.95%	Val Loss: 0.5196, Val Acc: 75.24%
Epoch 4/20: Train Loss: 0.4586, Train Acc: 79.53%	Val Loss: 0.4718, Val Acc: 78.89%
Epoch 5/20: Train Loss: 0.4558, Train Acc: 80.04%	Val Loss: 0.4753, Val Acc: 78.53%
Epoch 6/20: Train Loss: 0.4524, Train Acc: 80.00%	Val Loss: 0.4623, Val Acc: 79.69%
Epoch 7/20: Train Loss: 0.4393, Train Acc: 80.31%	Val Loss: 0.4555, Val Acc: 79.64%
Epoch 8/20: Train Loss: 0.4341, Train Acc: 80.59%	Val Loss: 0.4766, Val Acc: 79.24%
Epoch 9/20: Train Loss: 0.4353, Train Acc: 81.05%	Val Loss: 0.4455, Val Acc: 80.27%
Epoch 10/20: Train Loss: 0.4398, Train Acc: 80.16%	Val Loss: 0.4585, Val Acc: 80.58%
Epoch 11/20: Train Loss: 0.4344, Train Acc: 80.94%	Val Loss: 0.4499, Val Acc: 80.58%
Epoch 12/20: Train Loss: 0.4342, Train Acc: 80.99%	Val Loss: 0.4454, Val Acc: 80.98%
Epoch 13/20: Train Loss: 0.4356, Train Acc: 80.42%	Val Loss: 0.4406, Val Acc: 80.40%
Epoch 14/20: Train Loss: 0.4352, Train Acc: 80.73%	Val Loss: 0.4377, Val Acc: 81.38%
Epoch 15/20: Train Loss: 0.4307, Train Acc: 80.88%	Val Loss: 0.4439, Val Acc: 80.62%
Epoch 16/20: Train Loss: 0.4352, Train Acc: 80.97%	Val Loss: 0.4552, Val Acc: 80.49%
Epoch 17/20: Train Loss: 0.4361, Train Acc: 80.54%	Val Loss: 0.4583, Val Acc: 80.36%
Epoch 18/20: Train Loss: 0.4284, Train Acc: 80.90%	Val Loss: 0.4451, Val Acc: 81.16%
Epoch 19/20: Train Loss: 0.4219, Train Acc: 81.30%	Val Loss: 0.4444, Val Acc: 80.58%
Epoch 20/20: Train Loss: 0.4284, Train Acc: 81.21%	Val Loss: 0.4380, Val Acc: 80.93%

II.11. Utilisation des modèles predefinis RESET18 et VGG puis leurs entrainements.

```
=====
ENTRAÎNEMENT: Baseline Model VGG
=====
Epoch 1/20: Train Loss: 0.6236, Train Acc: 71.74% | Val Loss: 0.4639, Val Acc: 80.58%
Epoch 2/20: Train Loss: 0.5012, Train Acc: 77.99% | Val Loss: 0.4724, Val Acc: 81.02%
Epoch 3/20: Train Loss: 0.4960, Train Acc: 78.07% | Val Loss: 0.4692, Val Acc: 81.38%
Epoch 4/20: Train Loss: 0.4795, Train Acc: 78.00% | Val Loss: 0.4765, Val Acc: 80.53%
Epoch 5/20: Train Loss: 0.4804, Train Acc: 78.48% | Val Loss: 0.4540, Val Acc: 81.33%
Epoch 6/20: Train Loss: 0.4883, Train Acc: 78.01% | Val Loss: 0.5217, Val Acc: 81.07%
Epoch 7/20: Train Loss: 0.4776, Train Acc: 78.30% | Val Loss: 0.4598, Val Acc: 81.07%
Epoch 8/20: Train Loss: 0.4735, Train Acc: 78.92% | Val Loss: 0.4611, Val Acc: 81.64%
Epoch 9/20: Train Loss: 0.4826, Train Acc: 78.56% | Val Loss: 0.4721, Val Acc: 81.20%
Epoch 10/20: Train Loss: 0.4742, Train Acc: 79.01% | Val Loss: 0.4650, Val Acc: 81.78%
Epoch 11/20: Train Loss: 0.4672, Train Acc: 79.35% | Val Loss: 0.4574, Val Acc: 81.16%
Epoch 12/20: Train Loss: 0.4757, Train Acc: 78.69% | Val Loss: 0.4877, Val Acc: 81.42%
Epoch 13/20: Train Loss: 0.4636, Train Acc: 79.68% | Val Loss: 0.4792, Val Acc: 81.60%
Epoch 14/20: Train Loss: 0.4683, Train Acc: 79.60% | Val Loss: 0.5219, Val Acc: 81.47%
Epoch 15/20: Train Loss: 0.4771, Train Acc: 79.02% | Val Loss: 0.4990, Val Acc: 81.24%
Epoch 16/20: Train Loss: 0.4663, Train Acc: 79.70% | Val Loss: 0.5059, Val Acc: 81.02%
Epoch 17/20: Train Loss: 0.4716, Train Acc: 78.92% | Val Loss: 0.4798, Val Acc: 81.47%
Epoch 18/20: Train Loss: 0.4783, Train Acc: 79.27% | Val Loss: 0.4924, Val Acc: 80.89%
Epoch 19/20: Train Loss: 0.4791, Train Acc: 78.39% | Val Loss: 0.4903, Val Acc: 81.64%
Epoch 20/20: Train Loss: 0.4732, Train Acc: 79.51% | Val Loss: 0.4909, Val Acc: 81.91%
```

...



II.12. Utilisation d'un modèle Transformer VIT.

Principe et Fonctionnement

Le Vision Transformer adapte les Transformers du traitement du langage naturel à l'imagerie médicale en analysant une image comme une séquence de régions (patches) plutôt que comme une grille de pixels.

II.12. Utilisation d'un modèle Transformer VIT.

Étapes Principales du Modèle

- **Découpage en patches** : L'image 100×100 est découpée en 100 patches de 10×10 pixels.
- **Patch Embedding** : Chaque patch est projeté dans un espace de dimension 768 pour former un token.
- **Token de Classe (CLS)** : Un token spécial est ajouté pour résumer l'information globale de l'image.
- **Encodage de Position** : Des informations de position sont ajoutées afin de préserver la structure spatiale.
- **Blocs Transformer** : Comprenant une self-attention multi-têtes(12), une normalisation de couche et un réseau feed-forward avec connexions résiduelles et dropout. Les patches interagissent via le mécanisme de *self-attention*, permettant au modèle de capturer des relations globales entre régions du tissu.
- **Classification Finale** : La prédiction IDC / Non-IDC est réalisée à partir du token de classe uniquement.

II.12. Utilisation d'un modèle Transformer VIT.

ENTRAÎNEMENT: Vision Transformer

Learning rate: 0.001

Early stopping patience: 7

Using scheduler: True

Batch 20/165, Loss: 0.5714

Batch 40/165, Loss: 0.5788

Batch 60/165, Loss: 0.8642

Batch 80/165, Loss: 0.5561

Batch 100/165, Loss: 0.4578

Batch 120/165, Loss: 0.4405

Batch 140/165, Loss: 0.4225

Batch 160/165, Loss: 0.3554

Nouveau meilleur modèle: 78.31%

Epoch 1/20: Train Loss: 0.6334, Train Acc: 72.64% | Val Loss: 0.5203, Val Acc: 78.31% | LR: 0.001000 | Patience: 0/7

Batch 20/165, Loss: 0.4863

Batch 40/165, Loss: 0.5572

Batch 60/165, Loss: 0.6090

Batch 80/165, Loss: 0.3678

Batch 100/165, Loss: 0.6306

Batch 120/165, Loss: 0.5019

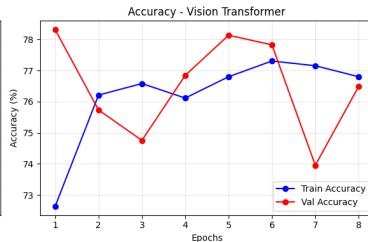
Batch 140/165, Loss: 0.6486

...

Epoch 8/20: Train Loss: 0.5060, Train Acc: 76.80% | Val Loss: 0.4972, Val Acc: 76.49% | LR: 0.001000 | Patience: 7/7

Early stopping déclenché après 8 epochs!

Meilleur modèle restauré avec une précision de validation de 78.31%



II.13 Métriques d'Évaluation Cruciales: Tableau de Performance Final (Synthèse)

--- UPDATED TABLEAU COMPARATIF DES PERFORMANCES (Ensemble de TEST) ---

	Loss (Test)	Accuracy (Test)	Recall (IDC Pos)	Precision (IDC Pos)	F1-Score (IDC Pos)	AUC-ROC
SimpleCNN1 (Baseline)	0.8424	0.7676	0.6693	0.8330	0.7422	0.8433
SimpleCNN2 (Avec dropout)	0.6894	0.7671	0.6018	0.8991	0.7210	0.8820
SimpleCNN2 (droup+Augmented)	0.3770	0.8462	0.8996	0.8129	0.8540	0.9175
SimpleCNN2 (Augmented+E)	0.3619	0.8551	0.9013	0.8251	0.8615	0.9235
SimpleCNN (Augmented+B+E)	0.3520	0.8573	0.8738	0.8460	0.8596	0.9230
PretrainedResNet	0.4358	0.8027	0.8409	0.7812	0.8099	0.8841
VGGAdapted	0.4909	0.8173	0.8604	0.7921	0.8249	0.8943
VisionTransformer	0.5032	0.7640	0.8036	0.7446	0.7730	0.8359

II.14 Interpretation en utilisant l'analyse Heatmap Grad-CAM

Objectif du Grad-CAM

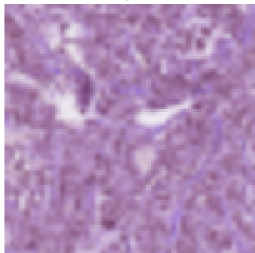
Le Grad-CAM (*Gradient-weighted Class Activation Mapping*) est utilisé pour interpréter les décisions du modèle en mettant en évidence les régions de l'image ayant le plus contribué à la prédiction finale.

II.14 Interpretation en utilisant l'analyse Heatmap Grad-CAM

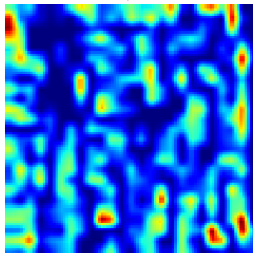
--- Grad-CAM Visualization ---

...

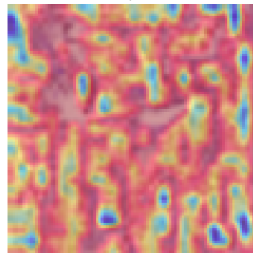
Original
True: 1, Pred: 1



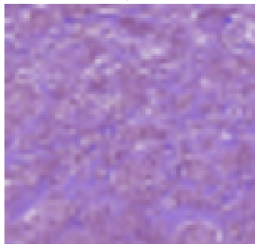
Heatmap
True: 1, Pred: 1



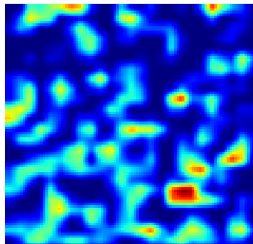
Grad-CAM Overlay
True: 1, Pred: 1



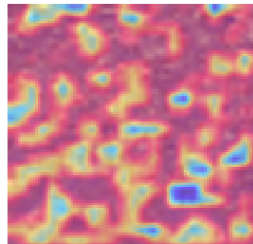
Original
True: 1, Pred: 1



Heatmap
True: 1, Pred: 1

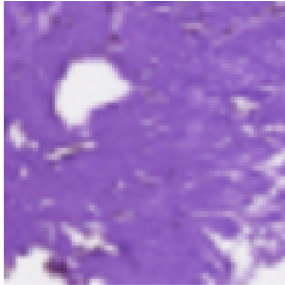


Grad-CAM Overlay
True: 1, Pred: 1

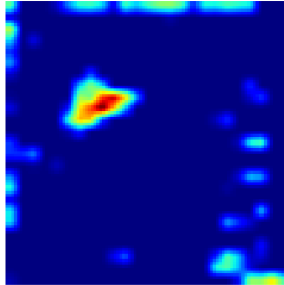


II.14 Interpretation en utilisant l'analyse Heatmap Grad-CAM

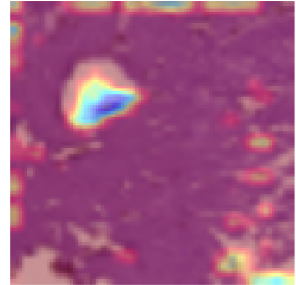
Original
True: 0, Pred: 0



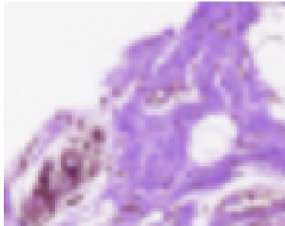
Heatmap
True: 0, Pred: 0



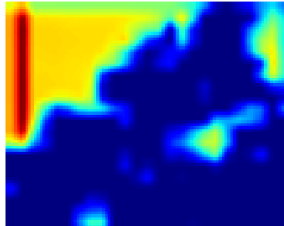
Grad-CAM Overlay
True: 0, Pred: 0



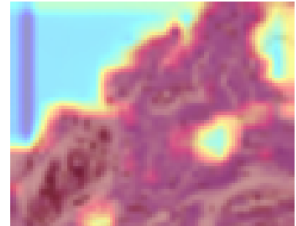
Original
True: 0, Pred: 0



Heatmap
True: 0, Pred: 0



Grad-CAM Overlay
True: 0, Pred: 0



II.14 Interpretation en utilisant l'analyse Heatmap Grad-CAM

On observe que La zone la plus chaude (rouge/jaune) est concentrée sur les amas cellulaires denses, irréguliers et chaotiques.

Le modèle est fiable. Il a appris à détecter la signature morphologique du carcinome canalaire infiltrant.

Le modèle a une bonne interprétabilité clinique car il utilise les mêmes indices qu'un pathologiste.

- 1 Introduction générale
- 2 I. Methodologie
- 3 II. Resultats
- 4 Conclusion

Synthèse du Projet et Modèle Vainqueur

- **Réalisation** : Le projet a permis de mettre en place un pipeline robuste de Deep Learning pour la classification de patches histopathologiques du cancer du sein (IDC), en traitant efficacement des défis majeurs tels que le déséquilibre des classes et le surapprentissage.
- **Meilleure Stratégie** : L'équilibrage des données, l'utilisation de la *data augmentation* et l'introduction d'un mécanisme d'*early stopping* se sont révélés déterminants pour améliorer la généralisation du modèle.

Performance Finale sur l'Ensemble de Test

- **Modèle Vainqueur : SimpleCNN2 (Augmented + Early Stopping)**
- **Métriques Clés (Classe IDC Positive) :**
 - AUC-ROC : **0.9235**
 - F1-Score : **0.8615**
- **Validation par Grad-CAM :** Les cartes d'activation confirment que le modèle se focalise sur des structures morphologiques pertinentes (amas cellulaires denses), renforçant la crédibilité et le potentiel d'utilisation clinique du modèle.